

核心问题重点回答

1. SGLang 的 RadixAttention/RadixCache 解决什么问题？

【在大模型推理中，多轮对话、Few-shot 提示以及 Agent 工作流等场景，通常包含大量相同或高度相似的提示词前缀。传统推理系统在每次处理新请求时，都会对这些相同前缀进行重复的 Prefill 计算，导致算力浪费与显存冗余。】

RadixCache 的核心目标是实现跨请求的前缀自动复用。 它采用**基数树 (Radix Tree)** 管理 KV Cache，能够动态识别不同请求间的公共前缀。当新请求到达时，系统自动执行最长前缀匹配，命中部分直接复用已有的 KV Cache，仅计算未命中的后缀。这极大减少了重复计算，显著提升了吞吐量并降低了首字延迟 (TTFT)。

2. SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

两者对应不同的处理阶段，一个是**内存管理机制**，另一个是**计算优化策略**：

- **Page-sized KV cache (物理存储+寻址机制)**：主要对应 Decode (逐字生成) 阶段的显存管理。它借鉴了**虚拟内存分页**思想，将 KV Cache 切分为固定大小的物理块进行按需分配，按需释放，以降低显存碎片。
- **Prefix reuse (逻辑复用+调度策略)**：主要对应 Prefill (输入计算) 阶段的前缀匹配。在 Prefill 执行前，调度器会查询 RadixCache，若存在可复用前缀，则直接跳过这部分计算，从而加速 Prefill 过程。

3. vLLM 的 PagedAttention 解决什么问题？

主要解决 **KV Cache 导致的显存碎片化和显存利用率低下**的问题。由于 KV Cache 的长度动态变化且难以预知，传统系统按最大长度预分配连续显存的做法会导致严重的内存碎片。PagedAttention 将 KV Cache 划分为固定大小的 Block，存储在非连续的物理显存中，并通过 Block Table 进行映射。配合**共享与写时复制 (Copy-on-Write) 机制**，显存利用率可提升至 90% 以上，从而**大幅提升系统吞吐量** (吞吐量提升约 2-4 倍)。

4. 它和 SGLang 的 RadixAttention/pagedKV 机制有什么联系与差异？

联系：

两者都采用了 **Page/Block 粒度的底层内存管理机制**，且均支持前缀共享，旨在解决大模型推理中的显存瓶颈。

差异：

- vLLM 的 PagedAttention 属于底层的内存管理层创新，侧重于提供通用、高吞吐的内存分配基础，其前缀共享通常需要较粗粒度的配置，偏向静态共享。
- SGLang 的 RadixAttention 则是在 PagedAttention 基础上，引入了前缀语义层的自动匹配。通过树状结构 (Radix Tree)，它能够自动处理任意长度、动态结构的复杂前缀共享，并配合缓存感知调度，实现更极致的缓存命中率。